

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Проектирование баз знаний»
на тему:

Интеллектуальный медицинский ассистент

БГУИР КП5 6-05-0611-03 046 ПЗ

Студент гр. 321701
Руководитель

В. А. Лукашов
Д. А. Сальников

Минск 2025

СОДЕРЖАНИЕ

Перечень условных обозначений	6
Введение	8
1 Анализ	10
1.1 Анализ предметной области	10
1.2 Постановка задачи	10
1.3 Требования к решению	11
1.4 Анализ аналогов	13
1.5 Анализ подходов к решению проблемы	17
1.6 Вывод	18
2 Проектирование	20
2.1 Анализ пользователей системы	29
2.2 Вывод	30
3 Разработка	31
3.1 Средства разработки	32
3.2 Демонстрация работы пользовательского интерфейса	33
3.3 Тестирование	35
3.4 Вывод	36
Список использованных источников	39

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

- БД — База данных;
БЗ — База знаний;
ГБ — Гигабайт;
ИИ — Искусственный интеллект;
ИТ — Информационные технологии;
КТ — Компьютерная томография;
Минздрав России — Министерство здравоохранения России;
МО — Машинное обучение;
НГУ — Новосибирский государственный интеллект;
ПГУ — Пензенский государственный университет;
ПО — Программное обеспечение;
СППВР — Системы поддержки принятия врачебных решений;
СУБД — Система управления базами данных;
ЭМК — Электронная медицинская карта;
AI (от англ. Artificial Intelligence) — Искусственный интеллект;
Aidoc (от англ. Artificial Intelligence Doctor) — Искусственный интеллект для врачей;
API (от англ. Application Programming Interface) — Программный интерфейс приложения;
AWS (от англ. Amazon Web Services) — Облачная платформа "Веб-сервисы от Amazon";
CDA (от англ. Clinical Document Architecture) — Клиническая архитектура документов;
ChatGPT (от англ. Generative Pre-trained Transformer) — Генерирующий предобученный трансформер для чата;
CRF (от англ. Conditional Random Fields) — Условные случайные поля;
CSS (от англ. Cascading Style Sheets) — Каскадные таблицы стилей;
DICOM (от англ. Digital Imaging and Communications in Medicine) — Цифровое изображение и связь в медицине;
DIMA (MD AI) (от англ. Medical Doctor Artificial Intelligence) — Доказательная медицина и Искусственный интеллект;
EDA (от англ. Event-driven Architecture) — Событийно-ориентированная архитектура;
EHR (от англ. Electronic Health Record) — Электронная медицинская запись;
FHIR (от англ. Fast Healthcare Interoperability Resources) — Ресурсы быстрого взаимодействия в сфере здравоохранения;
GDPR (от англ. General Data Protection Regulation) — Общий регламент по защите данных;

GPU (от англ. Graphics Processing Unit) — Графический процессор;

gRPC (от англ. Google Remote Procedure Call) — Удалённый вызов процедур от Google;

HL7 (от англ. Health Level Seven) — Стандарт обмена, управления и интеграции электронной медицинской информации «Седьмой уровень»;

HTML (от англ. HyperText Markup Language) — Язык гипертекстовой разметки;

IoT (от англ. Internet of Things) — Интернет вещей;

IT (от англ. Information Technologies) — Информационные технологии;

LLM (от англ. Large Language Model) — Большая языковая модель;

MedPlan (от англ. Medical Plan) — Медицинский план;

ML (от англ. Machine Learning) — Машинное обучение;

MQTT (от англ. Message Queuing Telemetry Transport) — Транспорт телеметрии с очередью сообщений;

NLP (от англ. Natural Language Processing) — Обработка естественного языка;

OpenCV (от англ. Open Source Computer Vision Library) — Библиотека алгоритмов компьютерного зрения, обработки изображений и численных алгоритмов общего назначения с открытым кодом;

PACS (от англ. Picture Archiving and Communication System) — Система архивирования и передачи изображений;

PathAI (от англ. Pathology + Artificial Intelligence) — Патология + Искусственный интеллект;

PMC (от англ. Public Medical Center) — Публичный медицинский центр;

RAG (от англ. Retrieval-Augmented Generation) — Генерация, дополненная поиском;

RAGPR (от англ. Retrieval-Augmented Generation-Based Physician Recommendation) — Рекомендация врачей на основе генерации с подкреплением поиском;

REST API (от англ. Representational State Transfer Application Programming Interface) — Передача репрезентативного состояния программного интерфейса приложения;

ROI (от англ. Return On Investment) — Возврат инвестиций;

SVM (от англ. Support Vector Machine) — Машина опорных векторов;

SQL (от англ. Structured Query Language) — Язык структурированных запросов;

TLS (от англ. Transport Layer Security) — Защита транспортного уровня;

WCAG (от англ. Web Content Accessibility Guidelines) — Руководство по доступности веб-контента;

ВВЕДЕНИЕ

Здравоохранение – это динамичная и многоуровневая система, включающая множество взаимосвязанных процессов, среди которых особое место занимает управление медицинскими данными и предоставление персонализированных рекомендаций медицинскому персоналу.[1] Эффективная организация этих процессов, контроль фармакологических назначений, оптимизация взаимодействия с медицинскими учреждениями и своевременное выявление рисков требуют применения современных цифровых решений, основанных на обработке данных, нормативных требованиях и передовых технологиях.[2]

В эпоху цифровизации ключевое значение приобретает создание и внедрение интеллектуальных систем поддержки медицинского персонала. Использование технологий *искусственного интеллекта (ИИ)* позволяет автоматизировать анализ данных, повышая их надёжность и экономическую эффективность. Такие системы, оснащённые централизованными *базами знаний (БЗ)*, способны агрегировать и анализировать информацию в реальном времени, что ускоряет принятие клинических решений и улучшает работу всей инфраструктуры.[3]

Для успешной реализации подобных систем необходимо разработать структурированную *БЗ*, включающую: данные о фармакологических препаратах и их взаимодействиях; характеристики медицинских изображений для анализа; информацию о возможных заболеваниях и методах их распознавания; сведения о взаимодействии с медицинскими учреждениями и их эксплуатационных особенностях. (Примечание: сбор и агрегация данных представляет собой сложную задачу, поскольку специализированные медицинские *базы данных (БД)* часто являются дорогими и труднодоступными. В проекте планируется использовать открытые источники и агрегированные датасеты для прототипа.) [4]

Такой подход позволяет не только повысить эффективность работы системы здравоохранения, но и минимизировать риски диагностических ошибок благодаря предиктивной аналитике. Формирование *БЗ* позволит систематизировать данные о ключевых аспектах здоровья: фармакологических рекомендациях, характеристиках медицинских изображений, стратегиях предотвращения заболеваний и особенностях взаимодействия с клиниками.[5]

БД также будет содержать: классификацию препаратов (включая взаимодействия и инструкции по применению); параметры диагностических процедур; перечень методов и инструментов для минимизации медицинских рисков.

Применение *ИИ-технологий* в такой системе выходит за рамки простого анализа данных — она способна прогнозировать состояние здоровья

на основе изображений, оптимизировать фармакологические рекомендации и предотвращать осложнения. Интеллектуальные алгоритмы автоматически адаптируют рекомендации в реальном времени, выявляют скрытые патологии на ранних стадиях и разрабатывают планы профилактического обслуживания.[6]

Перспективы развития системы связаны с внедрением технологий машинного обучения и прогнозирования. Это обеспечит переход от пассивного реагирования к активным стратегиям, снижая эксплуатационные издержки и продлевая срок службы оборудования. Например, предсказание ухудшения состояния пациента или выявление патологических изменений на ранних стадиях позволит заранее адаптировать лечебные процедуры. [7]

Цель работы – разработка интеллектуальной системы, предоставляющей рекомендации по фармакологии и распознаванию состояний по фото лица, с использованием *RAG-подхода* для анализа медицинских данных. В основу проектирования положены принципы модульности, безопасности и адаптивности, а также научный подход к анализу данных и поиску технических решений. [8]

Задачи исследования:

- изучение современных технологий в области медицинских ассистентов;
- формирование требований к интеллектуальной системе;
- проектирование архитектуры с учётом *RAG-методов*;
- оценка эффективности разработанного решения.

1 АНАЛИЗ

1.1 Анализ предметной области

Предметная область разрабатываемой системы связана с обработкой и интерпретацией медицинской информации в цифровом виде. В рамках проекта система рассматривается как вспомогательный инструмент, предназначенный для работы с пользовательскими данными и справочной медицинской информацией с постановкой диагнозов и назначением лечения.

Ключевым элементом предметной области является фармакологическая информация и клинические рекомендации. Система должна оперировать сведениями о лекарственных средствах, включая их назначение, ограничения по применению, возможные побочные эффекты и типовые схемы приёма. Данные данного типа представлены преимущественно в виде текстовых документов и рекомендаций, что требует их структурирования и приведения к формату, пригодному для машинной обработки и последующего поиска. В рамках проекта предполагается использование открытых источников медицинских данных и механизмов контекстного извлечения информации.

Вторая значимая часть предметной области связана с анализом визуальных признаков, наблюдаемых на изображениях лица пользователя. Рассматриваются только внешние характеристики, такие как изменение цвета кожных покровов, наличие отёчности, высыпаний или признаков усталости. Эти признаки могут указывать на возможные отклонения от нормы, однако результаты анализа носят исключительно справочный характер и не заменяют консультацию специалиста. С точки зрения системы данная область сводится к обработке изображений и сопоставлению выявленных признаков с заранее заданными шаблонами и описаниями.

Дополнительно предметная область включает особенности взаимодействия пользователя с системой. Вводимые данные могут быть представлены как в свободной форме (краткие описания самочувствия), так и в виде более формализованных медицинских текстов. Это накладывает требования на предварительную обработку входных данных, выделение ключевых сущностей и корректную интерпретацию пользовательских запросов в рамках выбранной медицинской терминологии.

1.2 Постановка задачи

В практической деятельности медицинских специалистов значительная часть времени тратится на работу с разнородной и слабо структурированной информацией. К таким данным относятся текстовые описания симптомов, эпикризы, результаты обследований, а также визуальные мате-

риалы, используемые для первичной оценки состояния пациента. Обработка этих данных во многих случаях выполняется вручную, что увеличивает нагрузку на специалиста и снижает скорость получения предварительных выводов.

Существующие программные решения, как правило, ориентированы на решение отдельных задач. Одни системы специализируются на анализе медицинских изображений, другие — на ведении документации или поиске справочной информации. При этом отсутствует единый инструмент, который позволял бы в рамках одного интерфейса работать как с текстовой медицинской информацией, так и с визуальными данными, учитывая их взаимосвязь в пределах одного клинического случая.

Отдельной проблемой является необходимость одновременного анализа различных форм представления данных. Текстовые источники, такие как жалобы пациента и медицинские записи, требуют извлечения ключевых медицинских сущностей и сопоставления их с базой знаний. Визуальные данные, в свою очередь, используются для выявления внешних признаков возможных отклонений и требуют применения отдельных методов обработки. Отсутствие комплексного подхода вынуждает специалистов использовать несколько разрозненных инструментов, что усложняет рабочий процесс.

Актуальность проекта обусловлена необходимостью создания прототипа гибридной системы, способной объединить анализ текстовой медицинской информации и визуальных данных в рамках одного программного решения. Такая система должна обеспечивать первичную обработку пользовательских запросов и формирование предварительных выводов, направленных на поддержку принятия решений на начальном этапе.

Целью проекта является разработка интеллектуального медицинского ассистента, предоставляющего два основных функциональных направления: обработку текстовых запросов для формирования рекомендаций по фармакотерапии с использованием локальной языковой модели и анализ изображений лица для выявления возможных состояний на основе визуальных признаков. В рамках проекта требуется реализовать модульную архитектуру с автоматическим определением типа входных данных и маршрутизацией запроса в соответствующий обработчик, а также обеспечить единый веб-интерфейс для взаимодействия пользователя с системой.

1.3 Требования к решению

Технологические аспекты решения.

Разрабатываемая система должна представлять собой прототип интеллектуального медицинского ассистента, ориентированный на практическую обработку пользовательских запросов. В рамках проекта система решает

две основные задачи: формирование фармакологических рекомендаций на основе текстовых данных и предварительный анализ состояния по изображению лица. Требования формулируются с учётом учебного характера работы и направлены на демонстрацию корректной архитектуры и взаимодействия компонентов, а не на создание промышленного программного продукта.

Система должна быть построена по модульному принципу, при котором отдельные подсистемы отвечают за приём пользовательских данных, обработку текстовых запросов, анализ изображений и формирование ответов. Такое разделение позволяет изолировать вычислительно сложные операции и упростить дальнейшее развитие проекта. Взаимодействие между модулями осуществляется через программные интерфейсы.

Клиентская часть системы реализуется в виде веб-приложения, доступного через браузер. Для этого используется современный JavaScript-фреймворк с поддержкой типизации, что упрощает разработку интерфейса и повышает надёжность клиентского кода. Серверная логика системы включает API-слой, отвечающий за обработку запросов, а также отдельные сервисы, реализованные на языке Python, предназначенные для работы с языковой моделью и алгоритмами анализа изображений.

Генерация текстовых ответов и рекомендаций выполняется с использованием локально развёрнутой языковой модели. Такой подход исключает передачу медицинских данных сторонним сервисам и обеспечивает автономную работу системы. Для повышения качества и обоснованности ответов используется механизм извлечения релевантной информации из внутренней базы знаний с последующей генерацией ответа на её основе. Это позволяет снизить вероятность некорректных или обобщённых рекомендаций по сравнению с использованием языковой модели без контекста.

Данные пользователей и результаты работы системы должны сохраняться в централизованном хранилище. В качестве СУБД используется PostgreSQL, обеспечивающая надёжное хранение структурированных данных. Система должна поддерживать регистрацию пользователей и контроль доступа. Аутентификация реализуется с применением хеширования паролей и токенов с ограниченным сроком действия. Передача данных между компонентами системы осуществляется по защищённому каналу. Развёртывание и запуск системы должны быть воспроизводимыми, что достигается использованием контейнеризации.[9]

Требования к эксплуатации системы ориентированы на использование в интерактивном режиме. Прототип должен корректно функционировать на стандартных персональных компьютерах без специализированных вычислительных ресурсов. Время обработки одного запроса, включая генерацию текстового ответа или анализ изображения, допускается в пределах нескольких десятков секунд. Архитектура системы должна допускать возможность перераспределения нагрузки, например вынос отдельных сервисов на от-

дельные вычислительные узлы при необходимости.

Функциональные характеристики системы.

Функциональность системы реализуется через единый интерфейс взаимодействия, построенный в формате диалога с пользователем. Все запросы, независимо от типа входных данных, обрабатываются в рамках чат-сессий, что упрощает работу с системой и позволяет сохранять контекст взаимодействия.

Обработка текстовых запросов предназначена для формирования рекомендаций по фармакотерапии. Пользователь вводит описание симптомов или состояния в произвольной форме. Полученный текст анализируется локальной языковой моделью, развёрнутой в системе, с целью интерпретации запроса и определения возможного направления терапии. В процессе обработки система выявляет недостающие параметры, необходимые для формирования корректных рекомендаций, и при необходимости инициирует уточняющий диалог. На основании собранной информации формируется текстовый ответ, содержащий перечень возможных лекарственных средств и общие рекомендации по их применению. Вся переписка сохраняется и доступна в рамках текущей сессии.

Анализ изображений реализован для обработки фотографий лица пользователя и получения предварительного заключения на основе визуальных признаков. Загруженное изображение автоматически передаётся в специализированный модуль анализа. Обработка выполняется поэтапно: на первом этапе проверяется наличие лица и качество изображения, достаточное для дальнейшего анализа. Далее извлекаются ключевые визуальные характеристики, такие как изменения цвета кожи, наличие отёчности или кожных проявлений. На завершающем этапе на основе полученного описания формируется текстовое заключение, указывающее на возможные отклонения или состояния, с которыми могут быть связаны выявленные признаки. В случае невозможности анализа пользователь получает уведомление о причине отказа.

Система поддерживает работу с несколькими независимыми диалогами. Каждый диалог представляет собой отдельный контекст взаимодействия и может использоваться для различных случаев или пользователей. История сообщений внутри диалога сохраняется в базе данных и восстанавливается при повторном обращении, что обеспечивает непрерывность работы. Доступ к функциональности системы осуществляется после прохождения процедуры регистрации и аутентификации пользователя.[10]

1.4 Анализ аналогов

Ada Health — это цифровая медицинская платформа, ориентированная на первичную оценку состояния пользователя на основе анализа симптомов.

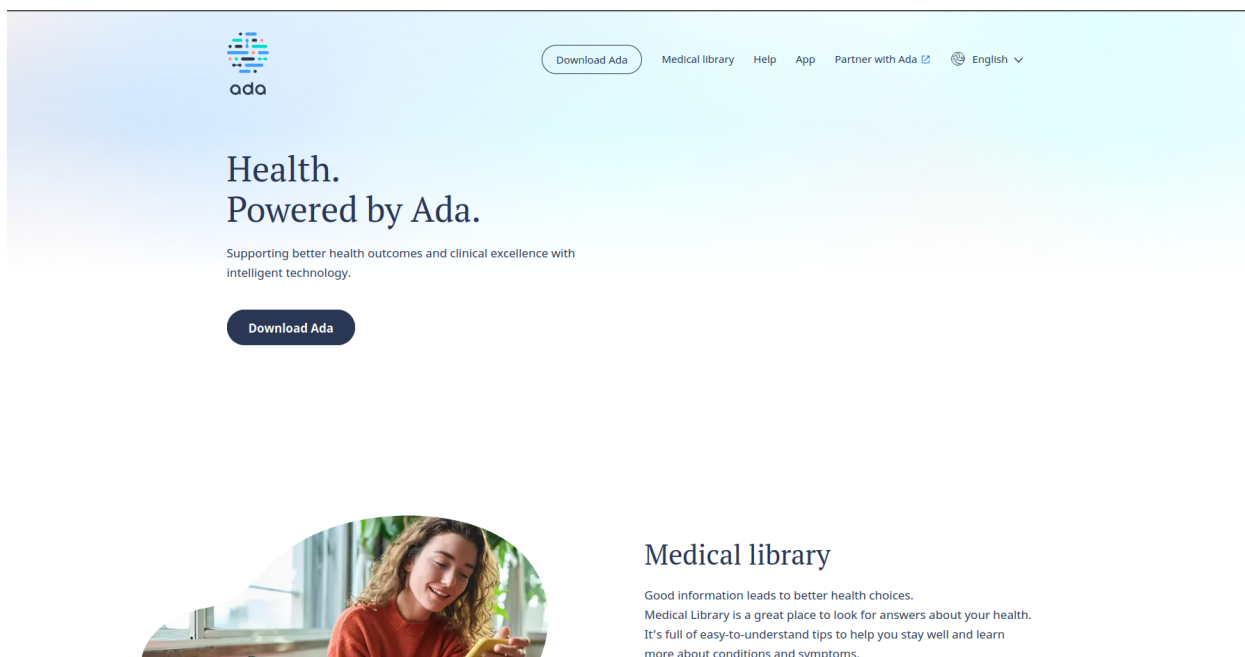


Рисунок 1.1 – Главная страница сайта Ada Health

Система реализована в виде мобильного и веб-приложения (на рисунке 1.1 представлено изображение главной страницы сайта Ada Health) и предназначена в первую очередь для использования пациентами без медицинского образования. Пользователь отвечает на последовательность вопросов о самочувствии, после чего система формирует перечень возможных состояний и рекомендации по дальнейшим действиям, таким как обращение к врачу или наблюдение за симптомами.[11]

К числу преимуществ Ada Health относится удобный пользовательский интерфейс и развитый механизм диалогового сбора информации. Система способна адаптировать последующие вопросы в зависимости от ранее полученных ответов, что позволяет уточнять симптомы и повышать релевантность результата. Платформа опирается на обширную медицинскую базу знаний и используется как инструмент самотриажа, снижая нагрузку на медицинские учреждения за счёт фильтрации некритичных обращений.

В то же время функциональные возможности Ada Health имеют ряд ограничений. Система ориентирована исключительно на текстовое взаимодействие и не поддерживает анализ визуальных данных, таких как фотографии пациента. Формируемые выводы носят обобщённый характер и не включают детализированные фармакологические рекомендации с учётом индивидуальных факторов, включая принимаемые препараты и возможные лекарственные взаимодействия. Кроме того, платформа функционирует как облачный сервис и не предоставляет возможности локального развёртывания, что ограничивает контроль над данными и затрудняет её применение в средах с повышенными требованиями к конфиденциальности.

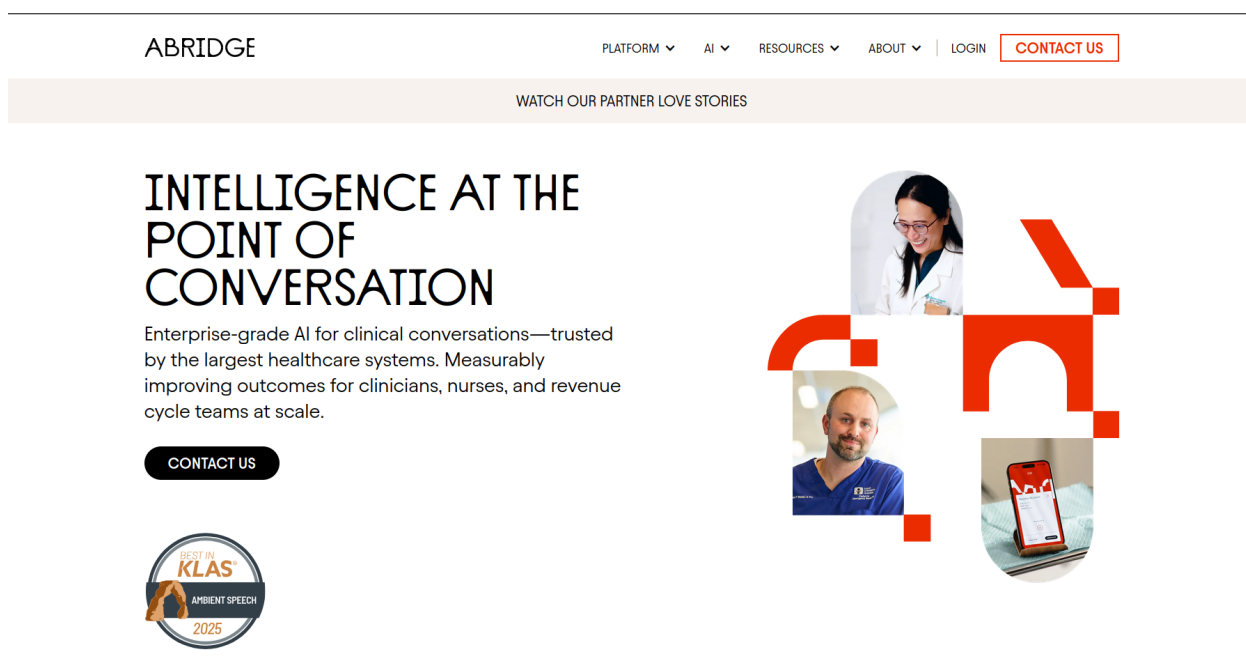


Рисунок 1.2 – Главная страница сайта Abridge

Abridge — это интеллектуальная платформа, предназначенная для автоматизации клинической документации и снижения административной нагрузки на врачей. Система используется в медицинских учреждениях для обработки диалогов между врачом и пациентом, преобразуя разговорную речь или текстовые заметки в структурированные медицинские записи. Основная область применения Abridge — поддержка ведения медицинской документации в процессе очных и дистанционных консультаций (на рисунке 1.2 представлено интерфейсное окно системы Abridge).[12]

Ключевым преимуществом Abridge является высокая степень автоматизации рутинных задач, связанных с заполнением клинических записей. Система способна выделять медицински значимые сущности из диалога, такие как симптомы, диагнозы, назначения и процедуры, и формировать на их основе структурированные отчёты. Это позволяет сократить время, затрачиваемое врачами на оформление документации, и повысить полноту медицинских записей. Платформа ориентирована на интеграцию с существующими электронными медицинскими картами и информационными системами.

Ограничения Abridge связаны с узкой специализацией системы. Платформа не предназначена для анализа визуальных данных и не поддерживает обработку изображений пациентов. Кроме того, Abridge не формирует самостоятельные медицинские рекомендации и не выполняет анализ фармакотерапии — её функциональность ограничивается фиксацией и структурированием уже озвученной информации. Система также ориентирована на облачную модель использования, что предполагает передачу данных

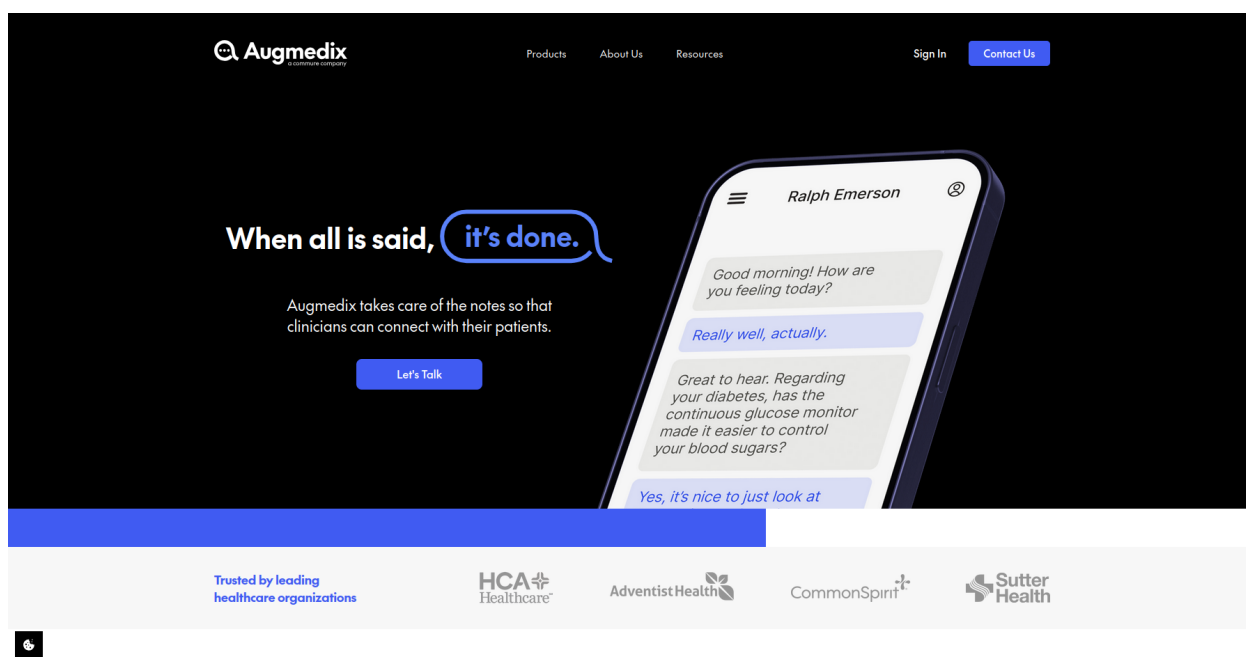


Рисунок 1.3 – Главная страница сайта Augmedix

на внешние серверы и может быть критичным в контексте требований к конфиденциальности медицинской информации.

Augmedix — это программная платформа, предназначенная для автоматизации процесса ведения медицинской документации в ходе приёма пациента (на рисунке 1.3 представлено изображение главной страницы сайта Augmedix). Система используется медицинскими специалистами для фиксации клинических данных в реальном времени и преобразования устной речи врача и пациента в структурированные записи, которые затем интегрируются в электронные медицинские карты. Основная цель Augmedix — снижение временных затрат на оформление документации и повышение эффективности работы врача.[13]

К преимуществам Augmedix относится ориентация на практическое использование в клинической среде и поддержка работы в режиме реального времени. Платформа способна автоматически выделять ключевые элементы медицинского диалога, включая симптомы, диагнозы, назначения и планы лечения. Это позволяет минимизировать ручной ввод данных и уменьшить вероятность пропуска важной информации. Augmedix активно интегрируется с существующими медицинскими информационными системами и используется в рамках устоявшихся клинических процессов.

В то же время функциональные возможности Augmedix ограничены задачами документирования. Система не выполняет аналитическую обработку медицинских данных и не формирует рекомендаций по терапии или диагностике. Анализ изображений пациентов и визуальных признаков в платформе отсутствует. Кроме того, решение ориентировано на облачную

инфраструктуру и корпоративное использование, что делает невозможным локальное развёртывание и снижает гибкость применения в учебных или исследовательских проектах с повышенными требованиями к контролю данных.

Сравнительный анализ.

Рассмотренные системы имеют узкую специализацию. Ada Health выполняет первичную оценку состояния пользователя на основе текстовой информации, формирует общие рекомендации, но не поддерживает анализ визуальных данных и не даёт персонализированных фармакологических выводов. Abridge автоматизирует ведение клинической документации, структурирует диалог врача и пациента, снижая административную нагрузку, но не анализирует терапию и визуальные данные. Augmedix обеспечивает фиксацию данных в реальном времени и интеграцию с медицинскими системами, минимизируя ручной ввод, однако также не выполняет аналитическую обработку и не формирует рекомендации по терапии.

Ни одна из платформ не сочетает анализ текстовых медицинских данных с формированием персонализированных рекомендаций и использованием RAG-подхода. Каждая система решает конкретные задачи: Ada Health — самотриаж, Abridge и Augmedix — автоматизация документации и поддержка клинической работы.

Соответствие данных аналогов к заданным требованиям можно увидеть в таблице 1.1:

Требование	Ada Health	Abridge	Augmedix
Обработка текстовых медицинских данных	✓	✓	✓
Поддержка фармакотерапии	×	×	×
Анализ визуальных данных	×	×	×
Интеллектуальная генерация клинических рекомендаций	×	×	×
Конфиденциальность и локальное хранение данных	×	×	×

Таблица 1.1 – Соответствие рассмотренных платформ требованиям проекта

1.5 Анализ подходов к решению проблемы

Для разработки интеллектуального медицинского ассистента необходимо выбрать подход, обеспечивающий обработку текстовых медицинских данных, генерацию рекомендаций и удобство использования. Ниже рассмотрены возможные варианты реализации системы.

Модульный локальный ассистент

Данный подход предполагает установку компонентов системы на локальных серверах медицинского учреждения. Основной акцент делается

на автономной работе и контроле над данными.

Преимуществами являются высокая безопасность, возможность интеграции с внутренними EHR и PACS, а также работа без постоянного подключения к интернету. Недостатками являются ограниченные возможности масштабирования и необходимость ручного обновления компонентов.

Подход подходит для организаций с повышенными требованиями к конфиденциальности, но не обеспечивает простого расширения функционала для новых пользователей.[5]

Гибридная облачно-локальная система

Этот вариант объединяет локальные и облачные компоненты. Чувствительные данные обрабатываются локально, а менее критичная информация и обновления — через облако.

Преимущества включают масштабируемость, централизованное обновление и гибкую интеграцию с существующими системами. Ограничениями являются повышенная сложность архитектуры и необходимость синхронизации между локальными и облачными модулями.

Подход обеспечивает баланс между безопасностью и функциональностью, но требует дополнительных ресурсов на поддержку инфраструктуры.[1]

Интерактивный голосовой ассистент для специалистов

Данный подход ориентирован на использование голосовых команд для поддержки работы врачей, автоматического заполнения заметок и генерации рекомендаций в реальном времени.

Достоинствами являются ускорение документации, снижение ручного ввода и возможность работы в реальном времени. Недостатками — ограниченный аналитический функционал, зависимость от локального оборудования и необходимость обучения распознаванию речи.

Этот вариант подходит для активной поддержки врачей в процессе приёма, но не может полностью заменить текстовые или визуальные методы анализа данных.

1.6 Вывод

Проведённый анализ показал, что существующие медицинские платформы решают узконаправленные задачи и не обеспечивают единой среды для обработки текстовой и визуальной информации. Системы либо фокусируются на анализе симптомов и самоотриаже, либо на автоматизации документации, при этом отсутствует интеграция данных разных типов и формирование персонализированных рекомендаций.

Сравнительный анализ выявил основные ограничения: отсутствие локальной обработки и контроля над данными, невозможность учёта индивиду-

альных факторов при формировании рекомендаций, а также ограниченные возможности аналитической обработки визуальных данных. Эти пробелы обосновывают необходимость разработки прототипа, который объединяет модули анализа текста и изображений, обеспечивает маршрутизацию запросов и формирует структурированные выводы для поддержки принятия решений.

Таким образом, результаты исследования подтверждают актуальность выбранного подхода — создание модульного интеллектуального ассистента с гибридной архитектурой, способного работать как с текстовыми, так и с визуальными данными. Полученные выводы формируют основу для дальнейшего развития системы и её возможной интеграции в рабочие процессы медицинских учреждений.

2 ПРОЕКТИРОВАНИЕ

Данный проект спроектирован как модульное клиент-серверное веб-приложение, обеспечивающее врачу быстрый доступ к рекомендациям по лекарственной терапии на основе современных клинических руководств. Архитектура системы построена вокруг взаимодействия нескольких агентов и включает три ключевых компонента, обеспечивающих скоординированную работу:

Клиентская часть — веб-интерфейс, реализованный с использованием Next.js 14 и TypeScript. Обеспечивает интуитивно понятный чат-интерфейс, отображение истории переписки, управление сессиями и адаптивность под мобильные устройства.

Серверная часть — REST API на базе FastAPI (Python 3.10), отвечающее за приём и валидацию запросов от клиента, управление контекстом сессии и координацию работы специализированных агентов.[14] Сервер формирует системный промпт для обработки запроса и инициирует вызов соответствующих агентов для анализа симптомов, формирования плана терапии и получения информации о препаратах.

Модуль взаимодействия с LLM — единая обёртка над локальными и внешними провайдерами языковых моделей, которая выполняет отправку запросов, получение и первичную обработку ответов. Модуль обеспечивает поддержку отказоустойчивости и возможность масштабирования при подключении нескольких провайдеров.

Система представляет собой веб-приложение для формирования медицинских рекомендаций, в котором взаимодействие осуществляется через чат-интерфейс. Определение сценария обработки выполняется автоматически и основывается исключительно на наличии или отсутствии фотографии в сообщении. Если пользователь отправляет только текст, система направляет данные в модуль фармакотерапии, реализованный на основе локальной LLM-модели, запускаемой через Ollama. Перед формированием терапевтического результата модель выполняет этап уточнения: определяет предполагаемое заболевание по описанным симптомам, выявляет возможные противопоказания и при необходимости запрашивает недостающие сведения. Диалоговый режим доступен только внутри текстовой фармакотерапии, что позволяет пользователю получать альтернативные варианты терапии или корректировать исходную информацию.[15]

При отправке изображения запрос автоматически направляется в многоагентную подсистему анализа лица. Работа подсистемы включает три последовательных этапа. На первом агент проверяет наличие лица на изображении и формирует текстовое описание визуальных признаков. На втором агент интерпретирует полученное описание и вычисляет вероятностное распределение возможных заболеваний. На третьем этапе генерируется

итоговый текстовый вывод, основанный на результатах предыдущих агентов. Если лицо на изображении отсутствует, дальнейшая обработка не выполняется, и пользователю возвращается соответствующее уведомление.

Серверная часть системы разработана Python 3.10 и обеспечивает обработку всех входящих запросов, маршрутизацию между агентами и формирование контекста для генерации рекомендаций. Центральным компонентом является оркестратор медицинских агентов, который координирует работу специализированных модулей: агент планирования лечения (TreatmentPlanner) анализирует текстовые данные, оценивает симптомы и предварительный диагноз, а при необходимости обращается к модулю MedicineProvider для получения информации о релевантных препаратах.[16]

Взаимодействие между агентами построено по сервисной модели: оркестратор направляет запросы, собирает результаты и формирует итоговую структуру терапевтического плана. MedicineProvider загружает локальный датасет лекарственных средств и предоставляет API для поиска препаратов по диагнозу или набору симптомов, обеспечивая отказоустойчивую работу при отсутствии данных или ошибках чтения.

Для обмена данными с клиентской частью используется REST API через HTTPS в формате JSON. Сервер сохраняет историю сообщений и контекст диалога в базе данных, что позволяет поддерживать непрерывность сессий и корректно обрабатывать уточняющие запросы пользователя. Модуль фармакотерапии и агенты работают локально в рамках сервера, а подсистема анализа изображений лица может быть развернута как независимый сервис, интегрируемый через Docker.[8]

На диаграмме вариантов использования интеллектуального медицинского ассистента(рисунок 2.1) изображена основная функциональность системы с точки зрения взаимодействия пользователя с системой. Актор «Пользователь» выполняет следующие ключевые действия:

- создание нового чата;
- регистрация;
- вход в аккаунт;
- выбор существующего чата из списка;
- просмотр истории чата;
- отправка текстового сообщения;
- отправка фото;
- получение рекомендаций.

Диаграмма компонентов серверной части медицинского ассистента (рисунок 2.2) отражает структуру основных модулей и их взаимодействие при обработке запросов пользователя. Выделены следующие ключевые компоненты:

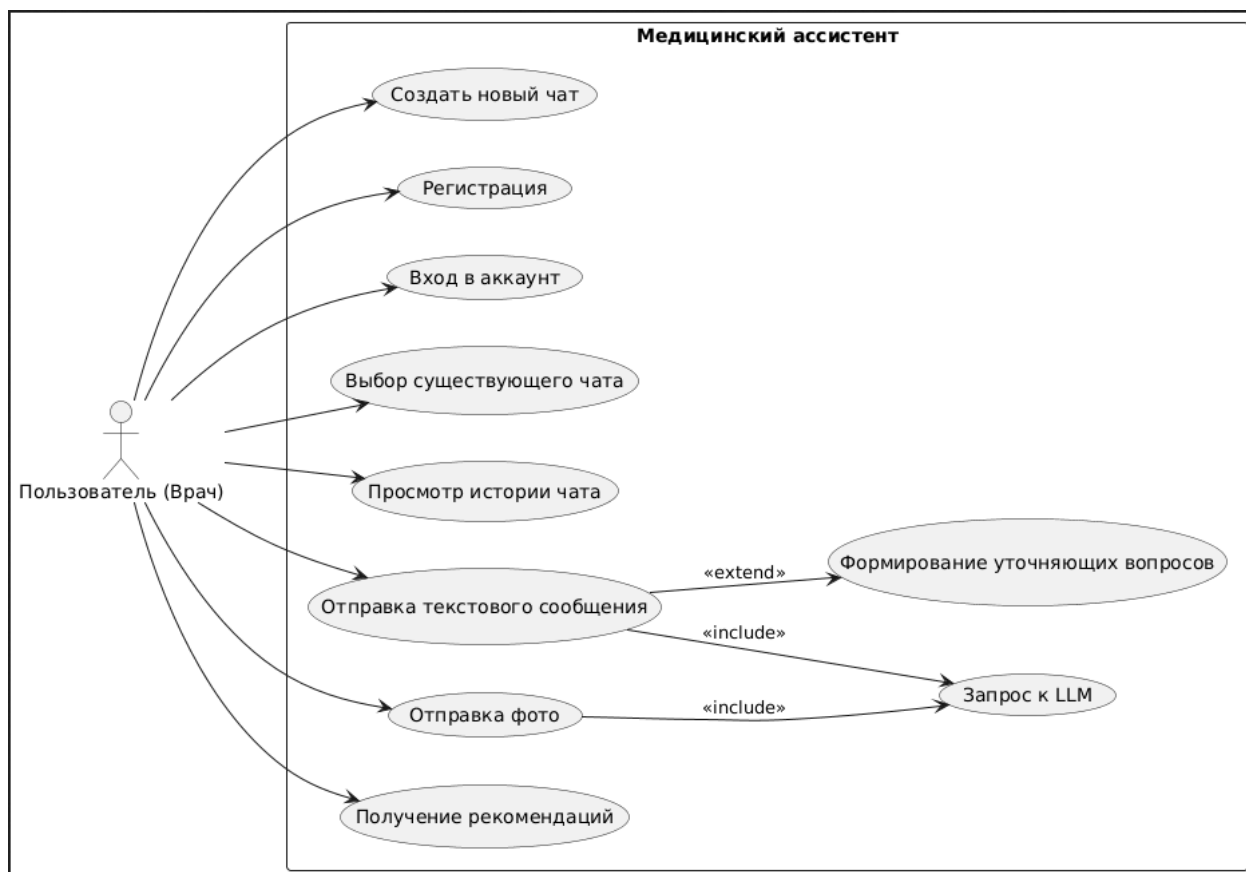


Рисунок 2.1 – Диаграмма вариантов использования интеллектуального медицинского ассистента

– Оркестратор медицинских агентов — центральный компонент, обеспечивающий маршрутизацию запросов, управление контекстом сессии и координацию работы всех агентов. Оркестратор принимает текстовые запросы, определяет последовательность вызова агентов и объединяет результаты в единый ответ.

– Агент планирования лечения (TreatmentPlanner) — анализирует симптомы и предварительный диагноз пользователя, формирует внутреннее представление клинической ситуации и инициирует запросы к другим агентам при необходимости.

– Модуль лекарственных препаратов (MedicineProvider) — обеспечивает поиск и предоставление информации о препаратах из локального датасета. Принимает запросы от TreatmentPlanner через оркестратор и возвращает структурированный список лекарственных средств с информацией о применении, заменителях и побочных эффектах.

– Модуль взаимодействия с LLM — обёртка для локальной или внешней большой языковой модели, выполняет генерацию терапевтических рекомендаций на основе сформированного контекста и системного промпта.

– API-контроллер — отвечает за приём REST-запросов от клиентской части, валидацию данных и передачу обработанных запросов оркестратору.

Взаимодействие компонентов построено по принципу слабой связности: оркестратор выполняет роль координатора, а отдельные агенты и сервисы могут быть подключены или отключены без изменения основной логики системы. Такая архитектура обеспечивает модульность, расширяемость и возможность интеграции новых аналитических модулей или источников данных в будущем.

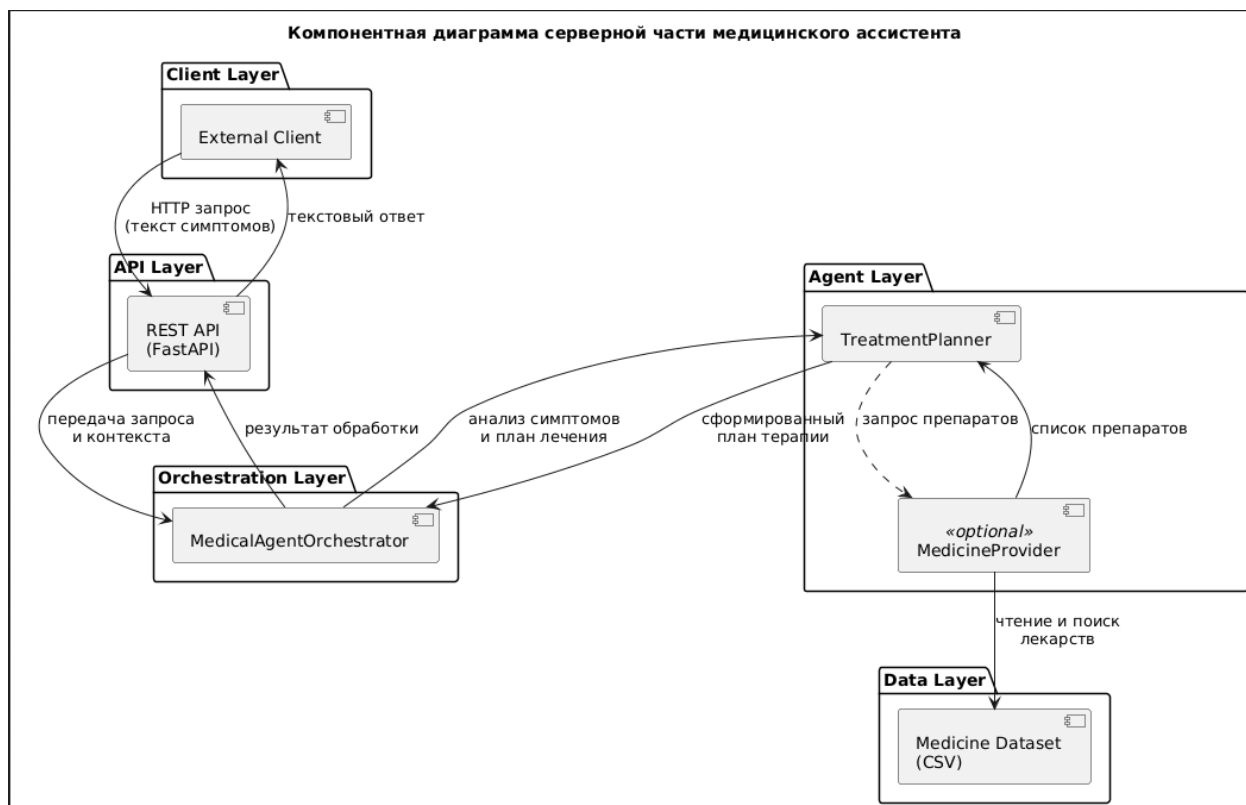


Рисунок 2.2 – Диаграмма компонентов серверной части медицинского ассистента

BRMN-диаграмма обработки сообщения с фотографией (рисунок 2.3) отображает полный процесс анализа изображений лица в системе. Процесс начинается с проверки наличия активной сессии чата. Если чат отсутствует, пользователь создаёт новый, после чего может отправлять сообщения.

При отправке сообщения пользователь прикрепляет фотографию (путём загрузки файла или через камеру устройства) и при необходимости добавляет текстовое пояснение. Система проверяет изображение на наличие лица. В случае, если лицо не обнаружено, пользователю сразу возвращается сообщение с уведомлением о невозможности обработки.

При успешном обнаружении лица инициируется работа многоагентной подсистемы. Первый агент оценивает качество изображения и создаёт подробное текстовое описание визуальных характеристик лица. Второй агент анализирует это описание и вычисляет вероятные медицинские состояния, связанные с визуальными признаками. Третий агент объединяет результаты

предыдущих шагов и формирует окончательный медицинский вывод.

Готовый результат сохраняется в базе данных как часть истории чата и одновременно отображается пользователю в интерфейсе, обеспечивая возможность дальнейшего взаимодействия и уточняющих вопросов.

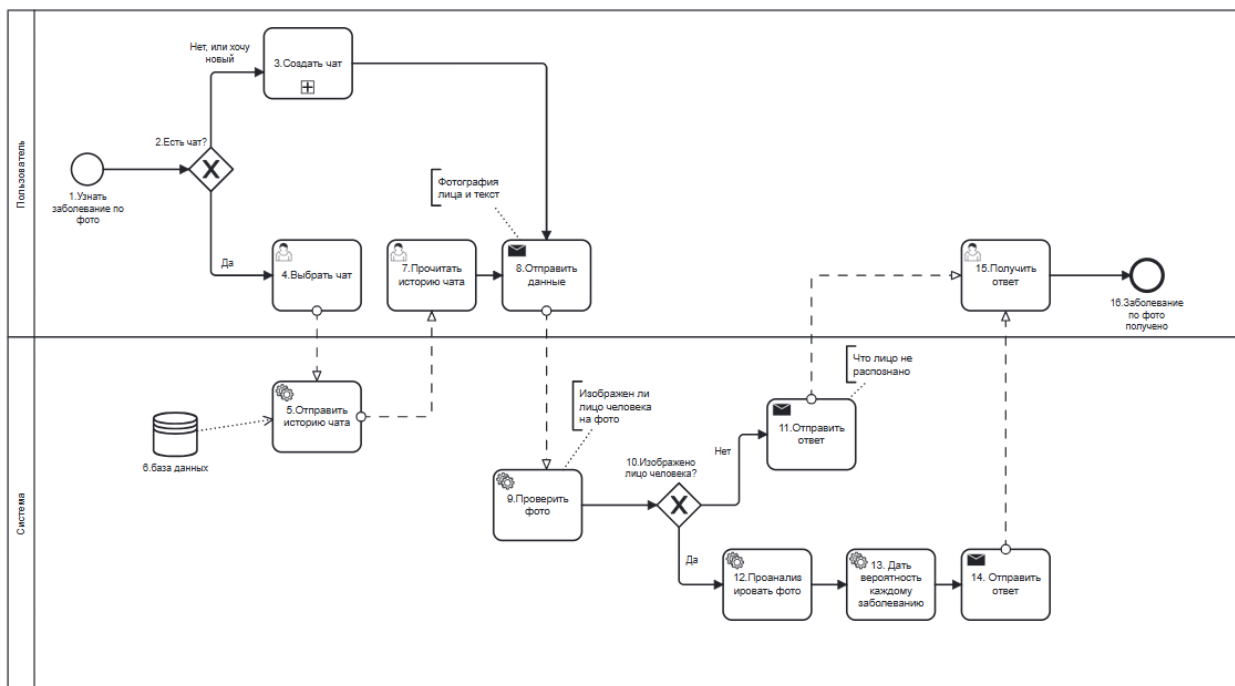


Рисунок 2.3 – BPMN-диаграмма обработки сообщения с фотографией лица

BPMN-диаграмма формирования фармакотерапии по текстовому запросу (рисунок 2.4) демонстрирует последовательность обработки сообщений, содержащих исключительно текст. После создания или выбора чата пользователь вводит описание симптомов или жалоб и отправляет сообщение на сервер.

На сервере локальная языковая модель (LLM, запущенная через Ollama) анализирует текст и определяет вероятный диагноз. Далее система автоматически проверяет наличие возможных противопоказаний и полноту исходных данных.

Если информации недостаточно (например, требуется уточнение возраста пациента, сопутствующих заболеваний, аллергий или текущих лекарств), система формирует уточняющий вопрос и направляет его пользователю. После получения ответа цикл обработки повторяется до тех пор, пока данные не станут достаточными для построения корректного плана терапии.

При наличии всех необходимых сведений LLM генерирует предварительный вариант фармакотерапии. Результат сохраняется в базе данных как часть истории чата и одновременно отображается пользователю в виде

структурированного текста. Процесс завершается либо выдачей окончательной рекомендации, либо продолжается в рамках диалога для уточнения дополнительных деталей по терапии.

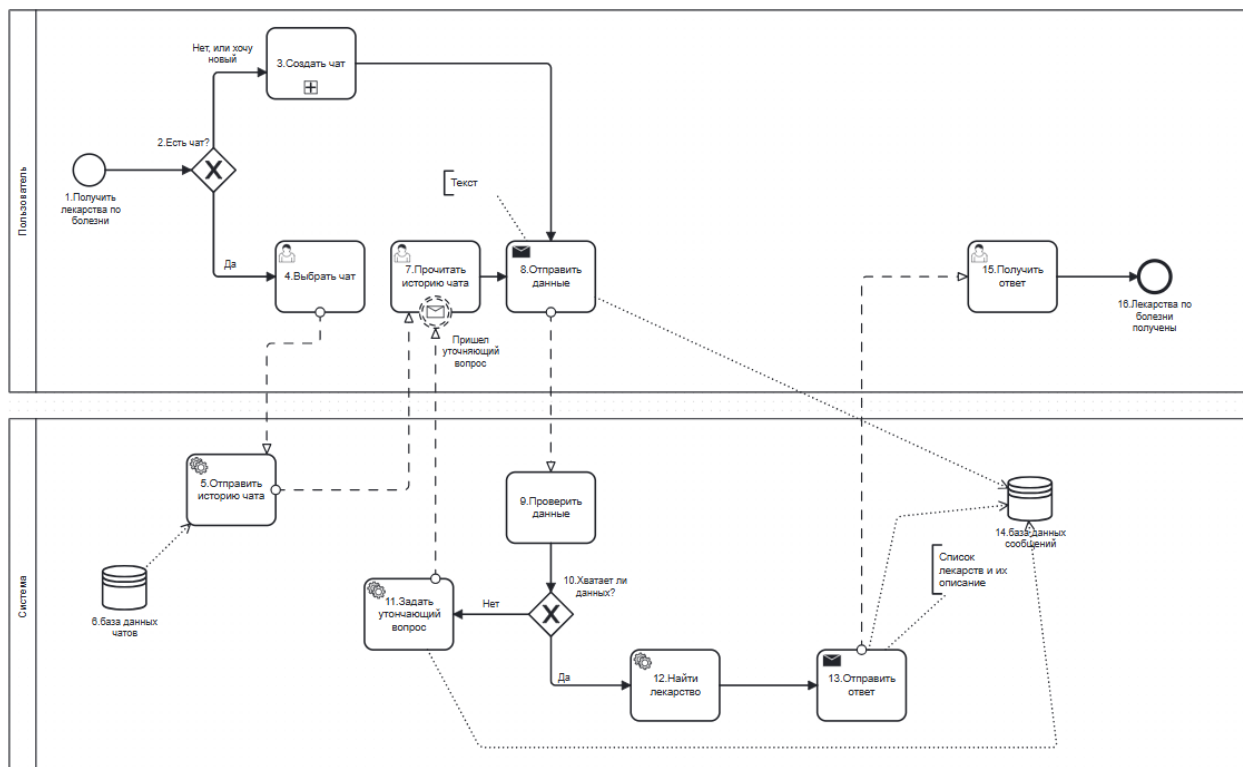


Рисунок 2.4 – BPMN-диаграмма формирования фармакотерапии по текстовому запросу

BPMN-диаграмма обработки медицинского запроса на сервере (рисунок 2.5) отражает внутренний цикл обработки сообщений, поступающих от пользователя. Процесс начинается с получения запроса через REST API и его валидации: проверяется корректность данных и наличие активного чата.

После валидации оркестратор медицинских агентов формирует единый контекст обработки и направляет данные к агенту планирования лечения (TreatmentPlanner). TreatmentPlanner анализирует симптомы и предварительный диагноз, определяет необходимость обращения к вспомогательным источникам и формирует внутреннюю структуру терапевтического плана.

При подключенном модуле MedicineProvider система отправляет сервисный запрос для получения списка релевантных лекарственных средств с информацией о применении, побочных эффектах и заменителях. Полученные данные интегрируются в план терапии. Если MedicineProvider недоступен или данных недостаточно, агент использует собственные алгоритмы формирования рекомендаций без привлечения внешнего датасета.

После завершения формирования терапевтического плана результат сохраняется в базе данных как часть истории чата. Затем сервер возвращает структурированный ответ клиенту для отображения в интерфейсе. Процесс может повторяться в рамках диалога для уточнения деталей, если информация изначально оказалась неполной или пользователь запросил дополнительные варианты терапии.

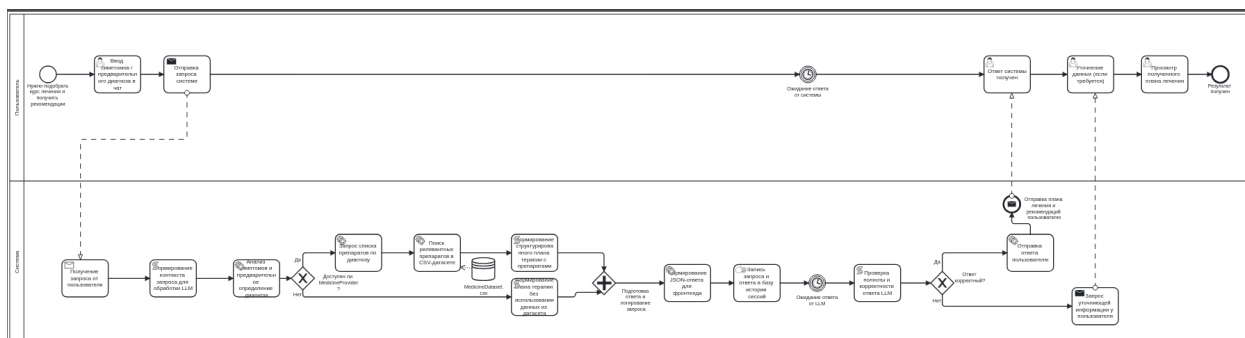


Рисунок 2.5 – BPMN-диаграмма обработки медицинского запроса на сервере

Диаграмма последовательности основных сценариев взаимодействия в чате (рисунок 2.6) отображает три ключевых потока обмена данными между пользователем, фронтендом и бэкендом.[17]

Первый поток — открытие существующей сессии чата. Пользователь выбирает чат из списка, после чего фронтенд отправляет GET-запрос к `/api/chat/id`. Сервер возвращает историю сообщений, которая отображается в интерфейсе, обеспечивая восстановление контекста диалога.

Второй поток — обработка сообщений с изображением лица. Пользователь прикрепляет фото к сообщению и отправляет его. Фронтенд формирует POST-запрос на `/api/chat/new/image`, передавая файл, текстовый контекст и идентификаторы пользователя и чата. Orchestration-service запускает последовательную работу агентов анализа лица: первый агент проверяет качество и формирует текстовое описание визуальных признаков, второй агент интерпретирует признаки и вычисляет вероятные диагнозы, третий агент агрегирует результаты и генерирует окончательный текстовый вывод. Итог сохраняется в истории чата и сразу отображается пользователю.

Третий поток — запрос фармакотерапии по тексту. Пользователь отправляет сообщение через POST `/api/v1/chat`. Pharma-service (локальная LLM через Ollama) анализирует текст и либо возвращает готовую терапевтическую схему, либо формирует уточняющий вопрос. В случае необходимости дополнительной информации (например, сопутствующие заболевания или аллергии) вопрос отображается во фронтенде как сообщение от ассистента, пользователь отвечает, и цикл повторяется до получения окончательной рекомендации.

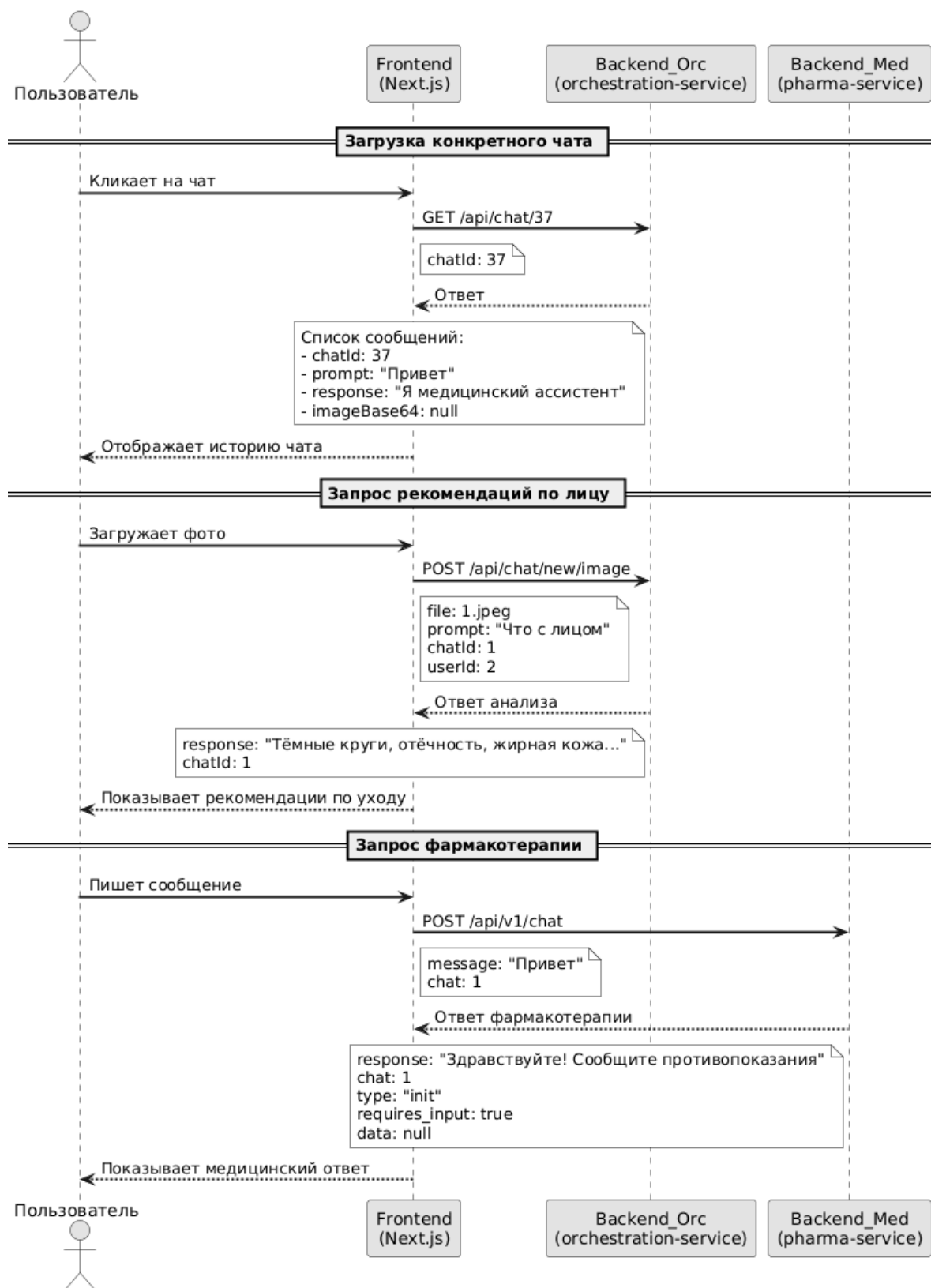


Рисунок 2.6 – Диаграмма последовательности основных сценариев работы чата

Диаграмма последовательности обработки текстового медицинского запроса (рисунок 2.7) демонстрирует чётко структурированный, мно-

гоуровневый процесс взаимодействия между пользователем и системой, включающий как логику анализа симптомов, так и принятие решений о фармакотерапии. Процесс можно разбить на три ключевых этапа: инициация запроса, анализ и планирование терапии, а также генерация итоговой рекомендации.[7]

Первый этап — инициация и передача запроса. Пользователь отправляет текстовый запрос, содержащий описание симптомов, через интерфейс. Этот запрос поступает на REST API (FastAPI), который выступает в роли шлюза, передавая входные данные и контекст (например, историю диалога или профиль пользователя) в центральный компонент системы — MedicalAgentOrchestrator. На этом этапе происходит первичная маршрутизация и подготовка данных для дальнейшей обработки.

Второй этап — анализ симптомов и планирование терапии. MedicalAgentOrchestrator делегирует задачу модулю TreatmentPlanner, вызывая метод `process(input_text, context)`. Внутри TreatmentPlanner выполняется анализ симптомов и формируется предварительный диагноз. Затем система переходит к ключевому ветвлению (`alt`), которое определяет стратегию подбора препаратов: если MedicineProvider доступен, система запрашивает у него список препаратов, соответствующих либо диагнозу, либо симптомам, используя метод `find_medicines_by_diagnosis()`. Для этого MedicineProvider обращается к внешнему источнику данных — Medicine Dataset (CSV) — и извлекает информацию о препаратах. Полученные данные возвращаются в TreatmentPlanner, где на их основе формируется план терапии; если MedicineProvider недоступен (например, при отсутствии актуальных данных или технической ошибке), система переключается на резервную стратегию: TreatmentPlanner самостоятельно формирует рекомендации без привлечения внешнего датасета, полагаясь на встроенные правила или логику, заложенную в его алгоритме.

Третий этап — завершение и возврат результата. По завершении анализа и планирования TreatmentPlanner возвращает результат обратно в MedicalAgentOrchestrator, который, в свою очередь, передаёт итоговый ответ через REST API на фронтенд. Пользователь получает текстовую рекомендацию — либо готовую схему лечения, либо, в случае необходимости, уточняющий вопрос для сбора дополнительной информации.

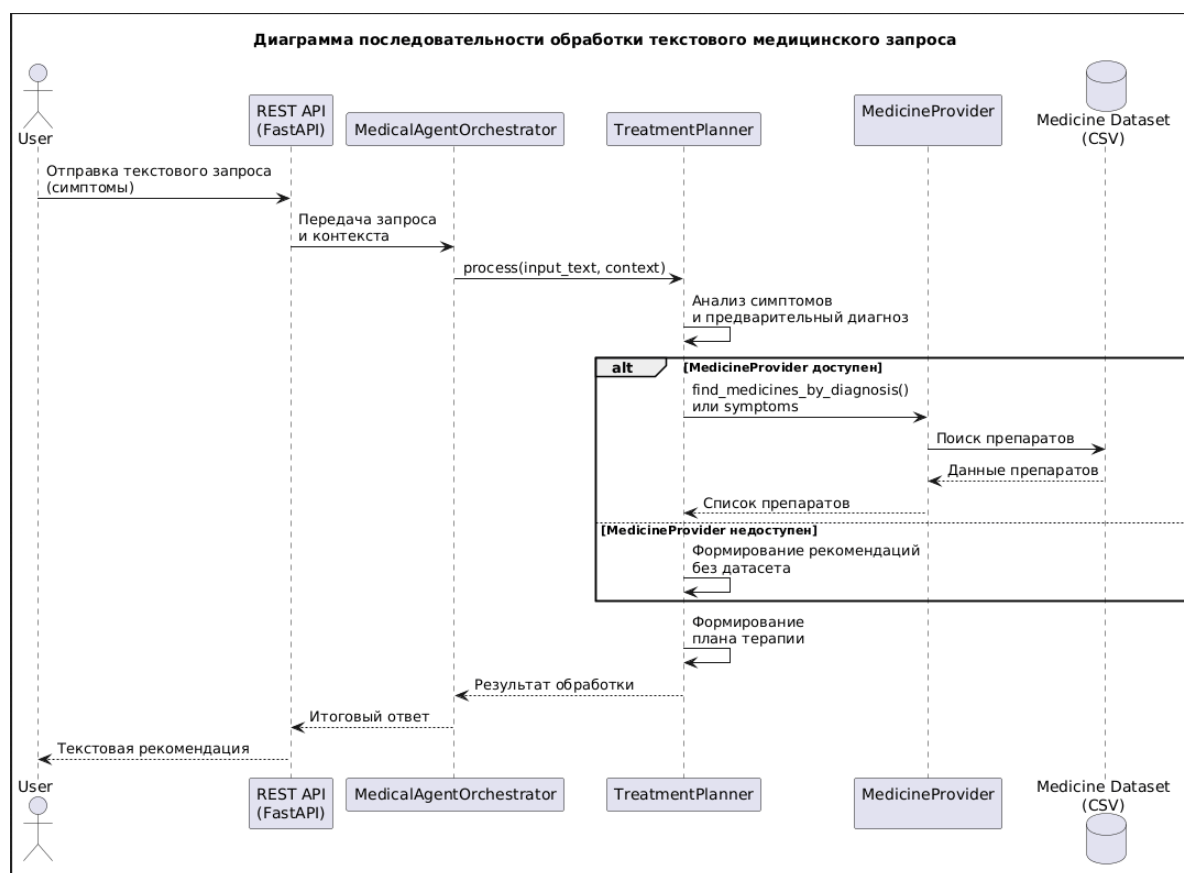


Рисунок 2.7 – Диаграмма последовательности обработки текстового медицинского запроса

2.1 Анализ пользователей системы

Для глубокого понимания механизмов взаимодействия пользователей с системой был проведён тщательный анализ их ролей, функций и потребностей, что позволило выявить ключевые аспекты, необходимые для обеспечения удобства, эффективности и адаптивности платформы. Этот анализ стал основой для проектирования системы, гарантирующей её соответствие реальным сценариям использования и ожиданиям различных категорий пользователей. В результате были определены два основных типа пользователей, чьи задачи и цели формируют фундамент для создания гибкого и функционального решения.[3]

Врач (конечный пользователь). Основной потребитель системы — медицинский специалист, который активно взаимодействует с ассистентом через веб-интерфейс, вводя название заболевания или клинической ситуации и получая рекомендации по лекарственной терапии. Такие пользователи ожидают высокой скорости ответа, точности дозировок и соответствия клиническим рекомендациям. Пример: врач вводит «острый бронхит у взрослого» и получает схему лечения с указанием препаратов, дозировок и

длительности курса.[2]

Администратор. Технический специалист, ответственный за настройку и обслуживание системы. Его задачи включают управление API-ключами к LLM, мониторинг нагрузки и обновление системного промпта при изменении клинических рекомендаций. Пример: после выхода приказа Минздрава администратор за несколько минут редактирует промпт, и уже следующий запрос врача получает обновлённую схему лечения.

2.2 Вывод

Предложенная архитектура серверной части описывает полный цикл обработки запросов пользователей с разделением логики на два независимых направления: текстовую фармакотерапию и анализ изображений лица. Автоматическое определение маршрута обработки обеспечивает единый сценарий взаимодействия, при этом внутренние механизмы генерации результата различаются в зависимости от типа данных.

Текстовые запросы анализируются локальной LLM-моделью с возможностью уточняющего диалога, формируя структурированные рекомендации и план терапии. Обработка изображений лица выполняется многоагентной подсистемой: агенты последовательно проверяют качество изображения, формируют текстовое описание визуальных признаков, вычисляют вероятные заболевания и генерируют итоговый вывод. Такое распределение задач обеспечивает модульность, отказоустойчивость и лёгкость расширения системы.

Анализ ролей пользователей подчёркивает различие целей: специалисты ориентированы на получение точных и обоснованных терапевтических схем, тогда как обработка изображений обеспечивает первичную оценку состояния. В совокупности архитектура полностью удовлетворяет функциональные и технические требования проекта, обеспечивая надёжную, масштабируемую и расширяемую серверную инфраструктуру.

3 РАЗРАБОТКА

Проектируемая система представляет собой серверную многоагентную подсистему интеллектуального медицинского ассистента, предназначенную для формирования предварительного плана лекарственной терапии на основе текстового описания симптомов и предполагаемого диагноза. Основной акцент в проектировании сделан на внутреннюю логику обработки медицинских запросов, координацию специализированных агентов и модульную архитектуру, допускающую расширение функциональности без изменения существующих компонентов.[5]

Центральным элементом системы является оркестратор медицинских агентов, который выполняет роль управляющего компонента и отвечает за маршрутизацию входных запросов, формирование контекста обработки и последовательный вызов специализированных агентов. Оркестратор не реализует медицинскую логику напрямую, а обеспечивает согласованное взаимодействие между агентами, что позволяет изолировать ответственность отдельных модулей и повысить устойчивость системы.

Ключевым агентом обработки текстовых запросов является агент планирования лечения (TreatmentPlanner). Он анализирует входной текст, содержащий симптомы или предварительный диагноз, формирует внутреннее представление клинической ситуации и определяет необходимость обращения к вспомогательным источникам данных. На основе полученного контекста агент формирует структуру плана терапии, которая в дальнейшем используется для генерации итогового ответа.

Для получения информации о лекарственных препаратах в системе реализован отдельный модульный агент MedicineProvider. Данный агент обеспечивает доступ к локальному датасету лекарственных средств и выполняет поиск препаратов по диагнозу или набору симптомов. MedicineProvider является опциональным компонентом системы и подключается динамически при инициализации оркестратора. При его отсутствии система продолжает корректно функционировать, используя только внутреннюю логику TreatmentPlanner без обращения к датасету препаратов.

Взаимодействие между TreatmentPlanner и MedicineProvider реализовано в виде сервисного запроса через оркестратор. При наличии активного MedicineProvider агент планирования лечения инициирует поиск релевантных препаратов и получает структурированный список лекарственных средств с информацией о применении, заменителях и побочных эффектах. Полученные данные интегрируются в общий план лечения. Если поиск не дал результатов или модуль недоступен, TreatmentPlanner формирует рекомендации без использования внешних источников лекарственной информации.

MedicineProvider загружает датасет препаратов при инициализации

и предоставляет унифицированный программный интерфейс для поиска и получения информации о лекарственных средствах. Все методы агента реализованы с учётом отказоустойчивости: при отсутствии датасета, некорректных входных данных или ошибках чтения возвращаются пустые результаты без генерации исключений. Это обеспечивает стабильную работу всей системы и изоляцию ошибок внутри отдельного модуля.

3.1 Средства разработки

Создание компонента для формирования лечебных рекомендаций осуществлялось с применением технологического стека, обеспечивающего эффективную обработку медицинской информации, устойчивое взаимодействие с другими элементами системы и соблюдение требований к производительности. Подбор средств выполнения обусловлен спецификой работы с диагнозами, необходимостью анализа ограничений к терапии и формированием персонализированных схем лечения.[4]

Язык программирования Python

Выбор Python в качестве базового языка разработки сервиса лечебных назначений объясняется его широким распространением в сфере создания медицинских информационных систем. Ключевые достоинства включают поддержку асинхронных операций, наличие специализированных библиотек для анализа данных (pandas) и обеспечения целостности информации (pydantic), а также простоту организации сетевого обмена. В медицинском контексте Python позволяет оперативно создавать надёжные решения для обработки клинических данных и генерации терапевтических схем.

Application Programming Interface

API представляет собой стандартизированный способ взаимодействия между системами, который позволяет получать данные и функциональность сторонних сервисов в удобном структурированном виде. Хотя API обеспечивают удобный доступ к данным, они имеют определенные недостатки, включая ограничения на количество запросов, платный доступ к некоторым функциям и необходимость сложной авторизации через ключи и токены.

Фреймворк FastAPI для веб-сервисов

Для построения API компонента назначения лечения выбран FastAPI, обеспечивающий высокоскоростную обработку запросов, автоматическое формирование документации OpenAPI и встроенную проверку корректности данных через Pydantic v2. Возможность асинхронного выполнения критически значима для одновременной обработки множества запросов на формирование лечебных рекомендаций, что гарантирует оперативность работы при обращении нескольких специалистов.[15]

Компонент формирования лечебных схем

Информация о фармакологических препаратах, их дозировках, ограничениях к применению и сопутствующих рекомендациях размещена в формате структурированного CSV-набора данных. Такой подход обеспечивает оперативное обновление терапевтических сведений, быстрый поиск соответствующих лечебных схем и лёгкое сопряжение с алгоритмами проверки совместимости. Данные организованы в соответствии с принципами медицинской классификации болезней и лекарственных средств.[6]

3.2 Демонстрация работы пользовательского интерфейса

На рисунке 3.1 показан стартовый экран пользовательского интерфейса медицинского ассистента. После открытия приложения пользователю отображается приветственное сообщение, информирующее о назначении системы и возможности получить помощь в анализе симптомов. В сообщении также указывается текущее время начала диалога, что позволяет отследить хронологию взаимодействия.

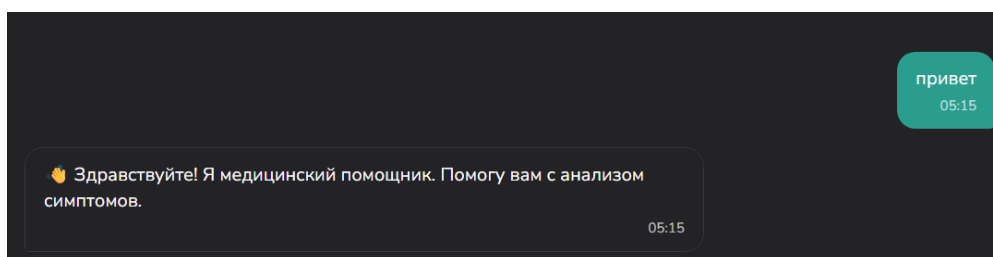


Рисунок 3.1 – Начальный экран интерфейса

На рисунке 3.2 представлен следующий этап взаимодействия — форма первичного медицинского опроса. Пользователю предлагается указать наличие факторов, которые могут повлиять на подбор терапии, включая лекарственные аллергии, хронические заболевания, регулярный приём медикаментов, а также беременность или период грудного вскармливания. При отсутствии указанных особенностей допускается выбор отрицательного ответа.

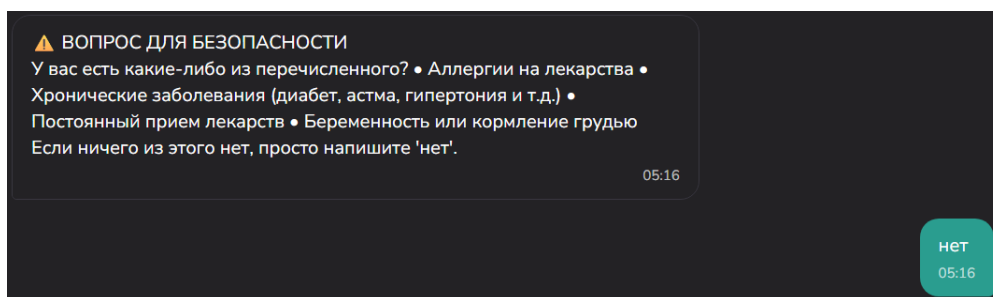


Рисунок 3.2 – Форма уточнения состояния здоровья пользователя

На рисунке 3.3 показан экран подтверждения полученного ответа. Система уведомляет пользователя о корректной обработке введённой информации и предлагает перейти к следующему этапу — описанию текущих жалоб и симптомов.

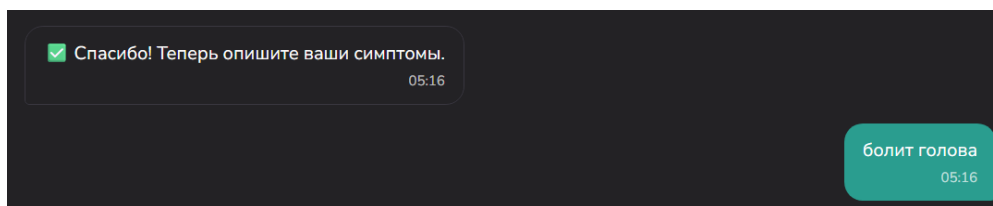


Рисунок 3.3 – Переход к этапу описания симптомов

Рисунок 3.4 иллюстрирует интерфейс ввода симптомов. Пользователю предоставляется возможность подробно описать своё состояние, включая характер болевых ощущений, их локализацию, продолжительность и сопутствующие проявления. В качестве демонстрационного примера приведено описание боли в горле. Поле ввода расположено в нижней части экрана и поддерживает ввод свободного текста.

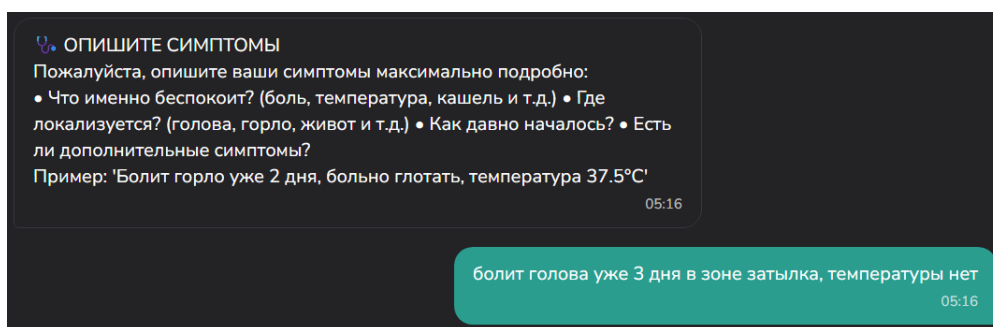


Рисунок 3.4 – Экран ввода и уточнения симптомов

На рисунке 3.5 отображён результат предварительного анализа, выполненного системой на основе введённых данных. Ассистент формирует гипотезу о возможном заболевании и выводит её в текстовом виде, подчёркивая предварительный характер результата.

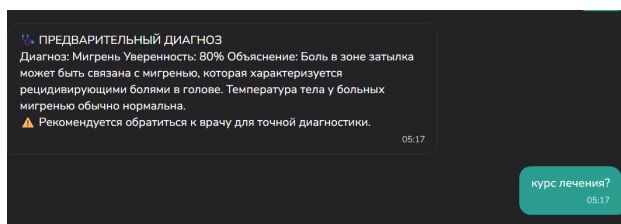


Рисунок 3.5 – Результат предварительного анализа состояния

На рисунке 3.6 представлен экран с развернутыми рекомендациями. Система отображает предложенный план терапии, включая перечень препаратов, способы их применения и общие рекомендации по дальнейшим действиям. Данный экран завершает основной сценарий взаимодействия пользователя с медицинским ассистентом.

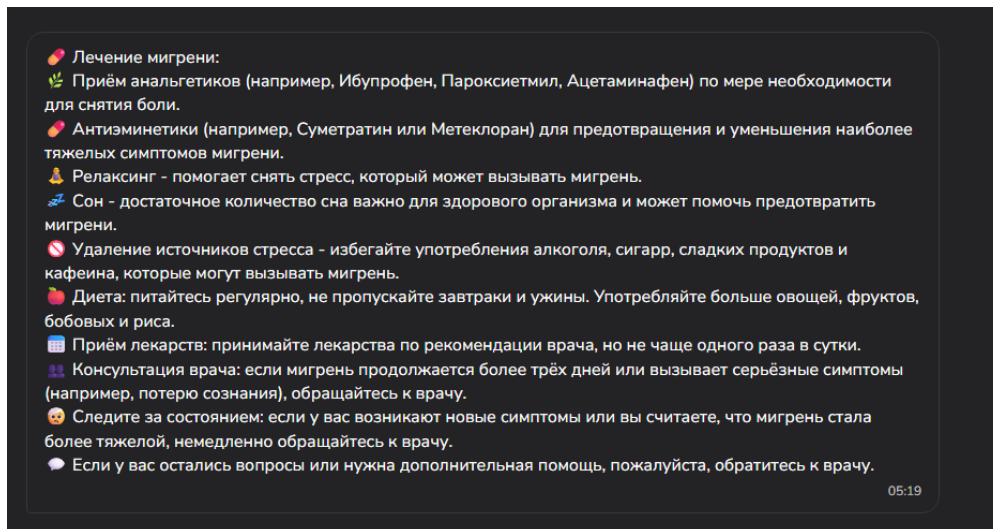


Рисунок 3.6 – Рекомендации и план лечения

3.3 Тестирование

Предложенная архитектура серверной части описывает полный цикл обработки запросов пользователей с разделением логики на два независимых направления: текстовую фармакотерапию и анализ изображений лица. Автоматическое определение маршрута обработки обеспечивает единый сценарий взаимодействия, при этом внутренние механизмы генерации результата различаются в зависимости от типа данных.

Текстовые запросы анализируются локальной LLM-моделью с возможностью уточняющего диалога, формируя структурированные рекомендации и план терапии. Обработка изображений лица выполняется многоагентной подсистемой: агенты последовательно проверяют качество изображения, формируют текстовое описание визуальных признаков, вычисляют вероятные заболевания и генерируют итоговый вывод. Такое распределение задач обеспечивает модульность, отказоустойчивость и лёгкость расширения системы.

Оркестратор медицинских агентов выступает центральным компонентом, координируя работу специализированных модулей, интегрируя результаты и формируя структурированный ответ для пользователя. Взаимодействие компонентов построено по принципу слабой связности, что позволяет добавлять новые агенты или источники данных без изменения основной логики системы.

Анализ ролей пользователей подчёркивает различие целей: специалисты ориентированы на получение точных и обоснованных терапевтических схем, тогда как обработка изображений обеспечивает первичную оценку состояния. В совокупности архитектура полностью удовлетворяет функциональные и технические требования проекта, обеспечивая надёжную, масштабируемую и расширяемую серверную инфраструктуру.

3.4 Вывод

В рамках раздела разработки была спроектирована серверная часть интеллектуального медицинского ассистента, ориентированная на обработку медицинских запросов и координацию работы специализированных агентов. Архитектура системы построена по модульному принципу, что позволило разделить ответственность между компонентами и обеспечить гибкость при расширении функциональности.

Ключевым элементом серверной логики является оркестратор медицинских агентов, обеспечивающий маршрутизацию запросов и управление контекстом обработки. Реализованные агенты планирования лечения и работы с лекарственными данными взаимодействуют через стандартизированные интерфейсы, что повышает отказоустойчивость системы и позволяет формировать терапевтические рекомендации даже при недоступности отдельных модулей.

Полученная архитектура системы обеспечивает структурированную обработку медицинской информации и создаёт основу для дальнейшего развития системы за счёт подключения новых агентов и аналитических модулей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана серверная часть интеллектуального медицинского ассистента, ориентированная на автоматизированную обработку медицинских запросов и формирование рекомендаций по лекарственной терапии. Реализованное решение демонстрирует практическое применение современных подходов к построению агентных систем и использованию языковых моделей для поддержки принятия решений в медицинской области.

Ключевой особенностью проекта стала модульная архитектура серверной части, основанная на использовании центрального оркестратора и набора специализированных агентов. Такое построение позволило чётко разделить ответственность между компонентами обработки запросов, анализа симптомов, планирования терапии и получения информации о лекарственных препаратах. Применение FastAPI в качестве серверного фреймворка обеспечило удобную реализацию REST-интерфейсов, высокую производительность и расширяемость системы.

Существенным результатом работы является разработка и интеграция агентного механизма обработки текстовых медицинских запросов. Агент планирования лечения выполняет анализ симптомов, формирует предварительную клиническую интерпретацию и, при необходимости, взаимодействует с модулем лекарственных средств. Использование отдельного компонента MedicineProvider, работающего с локальным датасетом препаратов, позволило реализовать поиск медикаментов по диагнозу и симптомам без жёсткой зависимости от языковой модели, что повышает устойчивость и контролируемость системы.

Интеграция локальной языковой модели, запускаемой через Ollama, показала эффективность в задачах интерпретации естественно-языковых запросов и генерации структурированных рекомендаций.[18] При этом архитектура системы предусматривает возможность уточняющего диалога и повторной обработки запроса при недостаточности исходных данных, что соответствует реальным сценариям медицинского консультирования. Отдельное внимание уделялось корректности формируемых ответов и исключению категоричных утверждений, что важно с точки зрения медицинской ответственности.

Реализованные BPMN- и UML-диаграммы наглядно отражают внутренние процессы серверной части, последовательность взаимодействия агентов и сценарии обработки пользовательских запросов. Это подтверждает логическую целостность архитектурного решения и упрощает дальнейшее сопровождение и развитие системы.

В целом разработанный бэкенд медицинского ассистента представляет собой устойчивую и расширяемую основу для интеллектуальной поддержки

врачей. Заложенные архитектурные принципы позволяют в дальнейшем подключать новые аналитические агенты, расширять источники медицинских данных и адаптировать систему под более сложные клинические сценарии без переработки базовой логики обработки запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Graph retrieval augmented large language models for facial phenotype associated rare genetic disease : [сайт]. — URL: <https://doi.org/10.1038/s41746-025-01955-x>, (дата доступа 16.11.2025).

[2] Enhancing medical AI with retrieval-augmented generation: A mini narrative review : [сайт]. — URL: <https://doi.org/10.1016/j.cmpb.2024.108123>, (дата доступа 16.11.2025).

[3] Harnessing RAG in Healthcare: Use-Cases, Impact, & Solutions : [сайт]. — URL: <https://hatchworks.com/blog/gen-ai/rag-for-healthcare/>, (дата доступа 16.11.2025).

[4] Artificial Intelligence/Machine Learning: The New Frontier of Clinical Pharmacology and Precision Medicine : [сайт]. — URL: <https://doi.org/10.1002/cpt.3198>, (дата доступа 16.11.2025).

[5] Dual retrieving and ranking medical large language model with retrieval augmented generation : [сайт]. — URL: <https://doi.org/10.1038/s41598-025-00724-w>, (дата доступа 16.11.2025).

[6] Искусственный интеллект в здравоохранении : [сайт]. — URL: <https://ai.minzdrav.gov.ru/>, (дата доступа 16.11.2025).

[7] Develop Secure, Reliable Medical Apps with RAG and NVIDIA NeMo Guardrails : [сайт]. — URL: <https://developer.nvidia.com/blog/develop-secure-reliable-medical-apps-with-rag-and-nvidia-nemo-guardrails/>, (дата доступа 16.11.2025).

[8] How Retrieval-Augmented Generation (RAG) Supports Healthcare AI Initiatives : [сайт]. — URL: <https://www.makebot.ai/blog-en/how-retrieval-augmented-generation-rag-supports-healthcare-ai-initiatives>, (дата доступа 16.11.2025).

[9] Обзор российских систем искусственного интеллекта для здравоохранения : [сайт]. — URL: <https://webiomed.ru/blog/obzor-rossiiskikh-sistem-iskusstvennogo-intellekta-dlia-zdravookhraneniia/>, (дата доступа 16.11.2025).

[10] SberMedAI : [сайт]. — URL: <https://sbermed.ai/>, (дата доступа 16.11.2025).

[11] Ada Health : [сайт]. — URL: <https://ada.com/>, (дата доступа 17.12.2025).

[12] Abridge : [сайт]. — URL: <https://www.abridge.com/>, (дата доступа 17.12.2025).

[13] Augmedix : [сайт]. — URL: <https://www.augmedix.com/>, (дата доступа 17.12.2025).

[14] FastAPI : [сайт]. — URL: <https://fastapi.tiangolo>, (дата доступа 16.11.2025).

[15] LangChain : [сайт]. — URL: <https://www.langchain.com>, (дата доступа 16.11.2025).

[16] Python : [сайт]. — URL: <https://www.python.org>, (дата доступа 16.11.2025).

[17] Incorporating Medical Assistants in eHealth Environments Using an Agentic RAG Approach : [сайт]. — URL: https://doi.org/10.1007/978-3-031-96235-6_12, (дата доступа 16.11.2025).

[18] Ollama : [сайт]. — URL: <https://www.ollama.com>, (дата доступа 16.11.2025).